



RAISE-Bot

— Hands-on ROS 2, AI Agents, and Greenhouse Robot Inspection



Calling a VLA via API

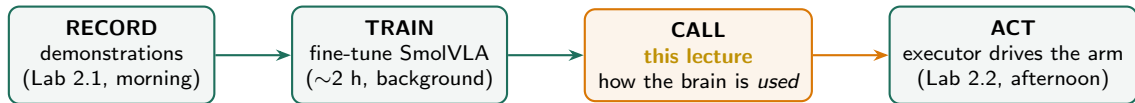
Student edition — where the VLA brain lives: one interface, two backends

Professor Anis Koubaa • 14–16 July 2026



Husky A200
Base

Where we are in Day 2

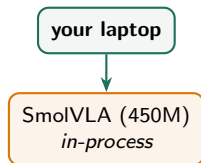


- Part 1 (dataset deck) answered: **how do we teach the model?**
- This part answers the *deployment* question: **once trained, how does the robot actually ask the model what to do?**

Definition — inference

Using a trained model to make predictions. For a VLA: one call = *(image, instruction, state)* in, *robot action* out — repeated many times per second while the arm moves.

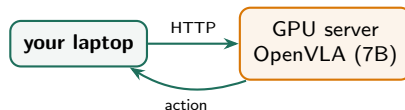
The question: where does the brain live?



LOCAL — the model is an object in your Python process.

- no network, works offline
- latency $\approx$ **tens of ms**
- needs your GPU (or patience)

This is **the** central deployment decision in real robotics — and today you can **measure** it.



REMOTE — the model sits behind an HTTP endpoint you call.

- any laptop can use a big model
- latency = round-trip (**50–200 ms** LAN)
- network down $\Rightarrow$ **no brain**

Design goal: the robot code must not care

Illustrative example. Imagine writing the executor twice:

```
if backend == "local":  
    action = policy.select_action(...)  
elif backend == "remote":  
    r = requests.post(url, ...)  
    action = decode(r, model_type)  
    # and per-model decoding...
```

```
client = make_vla_client()  
action = client.act(img, text, state)  
arm.send(action)
```

— identical for every backend.

The choice moves into **configuration**.

Two ideas make the right column possible:

- 1 a **common interface** — every backend answers `act` the same way;
- 2 a **canonical action contract** — every backend returns actions in the *same language*.

Why force a contract? Action spaces differ!

The *interface* unifies trivially. The *action semantics* do not — this is the gotcha that bites people:

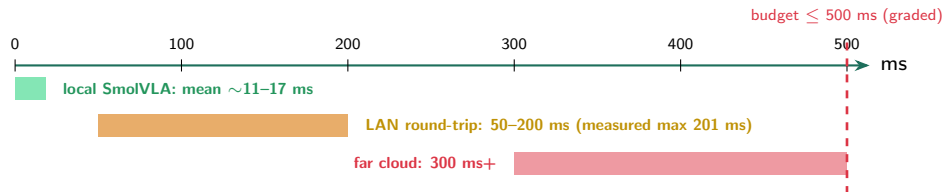
	SmolVLA (fine-tuned on your data)	OpenVLA (zero-shot)
Native output	UR5e joint targets	7-DoF end-effector deltas
Trained for	<i>your</i> greenhouse task	Bridge/RT tabletop data
Maps to our UR5e?	directly	needs un-normalize + IK adapter

- Each backend therefore owns a small adapter: `to_canonical()`.
- For the class, **both** backends serve *fine-tuned* checkpoints so both already speak UR5e-joint.
- Raw zero-shot OpenVLA is kept only as a demo of **why it flails** — a teaching moment, not the graded path.

Latency: the budget every robot call must meet

Definition — latency

Time from *asking* (`client.act` starts) to *knowing* (the action returns). While it passes, the robot moves on stale commands.

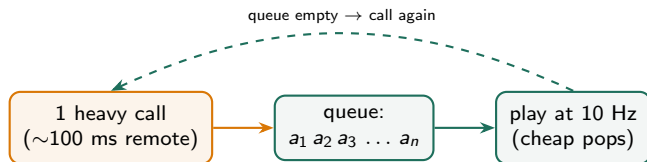


Where remote time goes: encode image → network out → **GPU inference** → network back → decode. On a LAN the network is cheap; across a continent it dominates.

Hiding latency: action chunking

Definition — action chunk

Instead of one action per model call, the policy predicts a short *sequence* of future actions (tens of steps) and plays them from an internal queue. Next calls pop the queue — only an *empty* queue costs a real model forward pass.



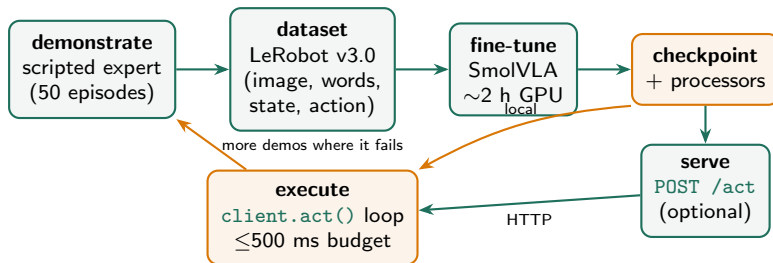
Example. At 10 Hz (a decision every 100 ms), calling a 150 ms-away server *every step* makes the arm stutter. A 30-step chunk turns one 150 ms call into **3 seconds** of smooth motion — the round-trip vanishes into the chunk. Bonus: chunks smooth *local* motion too.

Local vs remote: the trade-off you can measure today

Axis	Local SmolVLA (450M)	Remote OpenVLA (7B)
Latency	tens of ms, no network	round-trip + GPU queue
Capability	small but fine-tuned = sharp	16× larger
Needs local GPU	yes (or slow CPU)	no — any laptop
Network drops	keeps working	brainless
Update the model	re-deploy on every robot	swap server ckpt once
Fleet of robots	N copies, N GPUs	one server, N clients

- No universal winner — it's a **requirements** question: a warehouse fleet wants the server; a Wi-Fi-less field robot wants local.
- Day-2 result: the **small fine-tuned local** model beat the *big zero-shot* one. **Data beats parameters.**

The imitation workflow, end to end



The loop closes: where the policy fails, you **record more demonstrations** of exactly that case and fine-tune again — improvement comes from **data**, not from touching model code.

Takeaways

- 1 Deployment is a **first-class decision**: local (fast, offline, needs GPU) vs remote (any laptop, one shared server, needs network).
- 2 An **interface** (`VLAClient.act`) + a **canonical contract** (**Action**: 6 joints + gripper, clamped) make the executor backend-blind — swap brains with two env vars.
- 3 Action spaces are the real incompatibility (joints vs EE-deltas) — adapters + fine-tuned checkpoints fix it.
- 4 “Remote VLA” means **self-hosted** (`POST /act`) — no public VLA API exists, because robot latency is unforgiving.
- 5 Respect the **latency budget** (≤ 500 ms); **action chunking** hides round-trips — but stateful policies need `client.reset()` per episode.
- 6 Measured, not believed: local ~ 11 – 17 ms, LAN max 201 ms, 8/8 picks — and the small **fine-tuned** model beat the big zero-shot one. **Data beats parameters.**